Puppet Base Course

Day 2

Extensions of the Puppet Language

Day 2 — Agenda

Iterators and Lambdas

- `each`, `map`, `filter`, `reduce`
- Chaining iterators
- Practical patterns

Facts — System Information

- What are facts and where do they come from?
- Navigating the `$facts` hash
- Programming custom facts (Ruby + external)

Templates — Dynamic Configuration Files

- The template problem: code vs. content
- ERB templates — Ruby in your config files
- EPP templates — Puppet-native templating
- Inline templates

Separation of Code and Data

- Why mix code and data is an antipattern
- Hiera — a hierarchical database for configuration data
- Writing a Hiera hierarchy
- Automatic parameter lookup
- Merge strategies
- Inheritance and exceptions via the hierarchy

Part 1 — Iterators and Lambdas

Why Iterators?

Without iterators, repeated resource declarations are verbose and brittle:

```
# Repetitive — don't do this
package { 'vim':   ensure => installed }
package { 'curl':  ensure => installed }
package { 'htop':  ensure => installed }
package { 'tmux':  ensure => installed }
package { 'jq':    ensure => installed }
```

With iterators:

```
['vim', 'curl', 'htop', 'tmux', 'jq'].each |$pkg| {
  package { $pkg:
    ensure => installed,
  }
}
```

`each` — Iterate Over Arrays and Hashes

```
# Array iteration
$packages = ['nginx', 'php-fpm', 'mysql-client']

$packages.each |$package| {
  package { $package:
    ensure => installed,
  }
}

# Hash iteration — key and value
$users = {
  'alice' => { 'uid' => 1001, 'shell' => '/bin/bash' },
  'bob'   => { 'uid' => 1002, 'shell' => '/bin/zsh' },
}

$users.each |$username, $config| {
  user { $username:
    ensure => present,
    uid    => $config['uid'],
    shell  => $config['shell'],
  }
}
```

`map` — Transform a Collection

`map` returns a new array/hash without creating resources:

```
$raw_names = ['nginx', 'mysql', 'redis']

# Produce transformed data
$service_names = $raw_names.map |$name| { "service-${name}" }
# Result: ['service-nginx', 'service-mysql', 'service-redis']

# Build package resource titles with ensure
$pkg_ensure = $raw_names.map |$name| {
  { 'title' => $name, 'ensure' => 'installed' }
}

# Combine map + each
$raw_names.map |$n| { "/etc/systemd/system/${n}.service" }.each |$path| {
  file { $path:
    ensure => absent,
  }
}
```

`filter` — Select Elements Matching a Condition

```
$all_packages = ['nginx', 'apache2', 'vim', 'emacs', 'curl']

# Keep only packages starting with a specific letter
$a_packages = $all_packages.filter |$pkg| { $pkg =~ /^a/ }
# Result: ['apache2']

# Filter facts to find network interfaces that are up
$active_interfaces = $facts['networking']['interfaces'].filter |$iface, $data| {
  $data['ip'] =~ /\d+\.\d+\.\d+\.\d+/
}

# Install only packages not already excluded
$excluded = ['telnet', 'rsh-client']
$safe_packages = $all_packages.filter |$pkg| { !($pkg in $excluded) }
```

`reduce` — Fold a Collection to a Single Value

```
# Sum an array of numbers
$total = [1, 2, 3, 4, 5].reduce(0) |$memo, $value| { $memo + $value }
# Result: 15

# Build a combined string
$all_pkgs = ['nginx', 'vim', 'curl'].reduce('') |$memo, $pkg| {
  "${memo}${pkg} "
}
# Result: 'nginx vim curl '

# Build a hash from an array
$packages = ['nginx', 'vim', 'curl']
$pkg_hash = $packages.reduce({}) |$memo, $pkg| {
  $memo + { $pkg => 'installed' }
}
# Result: {'nginx' => 'installed', 'vim' => 'installed', 'curl' => 'installed'}
```

Chaining Iterators

Puppet supports method chaining — iterators can be composed:

```
# From a list of packages, keep only those not present, then install them
$desired  = ['nginx', 'curl', 'htop', 'telnet']
$excluded = ['telnet']

$desired
  .filter |$pkg| { !($pkg in $excluded) }
  .each   |$pkg| {
    package { $pkg: ensure => installed }
  }
```

```
# Build vhost config paths from a hash of sites
$sites.map |$name, $_| { "/etc/nginx/sites-available/${name}" }
      .each |$path| {
        file { $path: ensure => absent }
      }
```

`Integer.times` and `range`

```
# Create N numbered resources
Integer[1, 5].each |$i| {
  file { "/opt/worker/worker-${i}":
    ensure => directory,
    owner  => 'app',
  }
}

# range() function
range(1, 5).each |$n| {
  user { "worker${n}":
    ensure => present,
    uid    => 2000 + $n,
  }
}
```

Part 2 — Facts

What Are Facts?

Facts are pieces of information Facter collects about the managed node before the catalog is compiled:

- Operating system name, version, and family
- IP addresses and network interfaces
- Memory, disk, and CPU information
- Hostname and FQDN
- Virtual or physical, cloud provider, AWS region
- Kernel version

The server receives facts with every agent run, stores them in PuppetDB, and makes them available during catalog compilation as the `$facts` hash.

Navigating the `$facts` Hash

```
# OS information
$facts['os']['name']          # 'Ubuntu'
$facts['os']['release']['full'] # '22.04'
$facts['os']['family']        # 'Debian'

# Network
$facts['networking']['fqdn']          # 'web01.example.com'
$facts['networking']['ip']            # '10.0.1.100'
$facts['networking']['hostname']      # 'web01'

# Hardware
$facts['processors']['count']         # 4
$facts['memory']['system']['total']   # '7.59 GiB'

# Virtual
$facts['virtual']                     # 'kvm' or 'physical'
$facts['cloud']                       # provider info if available
```

Exploring Facts on a Node

```
# List all top-level facts
facter

# Query a specific fact
facter os
facter networking.fqdn
facter processors.count

# Output as JSON (useful for automation)
facter -j

# Show all values including structured facts
facter --show-legacy
```

Trusted Facts

Puppet also provides **trusted facts** — facts derived from the agent's certificate, which cannot be spoofed by the agent:

```
# These come from the SSL certificate — cannot be faked
$trusted['certname']        # agent01.example.com
$trusted['domain']          # example.com
$trusted['hostname']        # agent01
$trusted['extensions']      # custom OIDs embedded in the cert
```

> Use `$trusted['certname']` instead of `$facts['networking']['fqdn']` for security-sensitive decisions (node classification, firewall rules, access control).

Using Facts in Manifests

```
# Branch on OS family for package names
case $facts['os']['family'] {
  'RedHat': { $apache = 'httpd' }
  'Debian': { $apache = 'apache2' }
  default:  { fail("Unsupported family") }
}

package { $apache: ensure => installed }

# Set NTP server based on datacenter (encoded in fqdn convention)
if $facts['networking']['fqdn'] =~ /\.dc1\./ {
  $ntp_server = 'ntp1.dc1.example.com'
} else {
  $ntp_server = 'ntp1.dc2.example.com'
}
```

Custom Facts — Ruby

Place Ruby custom facts in `<module>/lib/facter/`:

```ruby
# modules/mymodule/lib/facter/app_version.rb

Facter.add('app_version') do
  setcode do
    # Run a command and return its output
    Facter::Core::Execution.execute('/opt/myapp/bin/myapp --version')
  end
end
```

```ruby
# Structured fact — returns a hash
Facter.add('app_info') do
  setcode do
    {
      'version' => Facter::Core::Execution.execute('/opt/myapp/bin/myapp --version'),
      'config'  => '/etc/myapp/config.yaml',
      'running' => system('systemctl is-active --quiet myapp'),
    }
  end
```

Custom Facts — External Facts

External facts are simple scripts or files — no Ruby required:

```
# modules/mymodule/facts.d/datacenter.sh
#!/bin/bash
echo "datacenter=dc1"

# Or return structured data as JSON
echo '{"datacenter": "dc1", "rack": "A12"}'
```

```
# YAML external fact
# modules/mymodule/facts.d/site_info.yaml
---
datacenter: dc1
rack: A12
environment_tier: production
```

> External facts are placed in `/etc/puppetlabs/facter/facts.d/` on the agent, or distributed via modules under `facts.d/`

Using Custom Facts

After deploying the module, the custom fact is available immediately:

```
# Use in a manifest
if $facts['datacenter'] == 'dc1' {
  $ntp_server = 'ntp.dc1.example.com'
} else {
  $ntp_server = 'ntp.dc2.example.com'
}

file { '/etc/ntp.conf':
  ensure  => file,
  content => "server ${ntp_server}\n",
}

# Structured fact access
$app_ver  = $facts['app_info']['version']
$app_conf = $facts['app_info']['config']
```

Part 3 — Templates

The Template Problem

Configuration files have two components:

1. **Structure** — the syntax (nginx.conf, sshd_config, haproxy.cfg…)
2. **Values** — the data that varies per environment or node

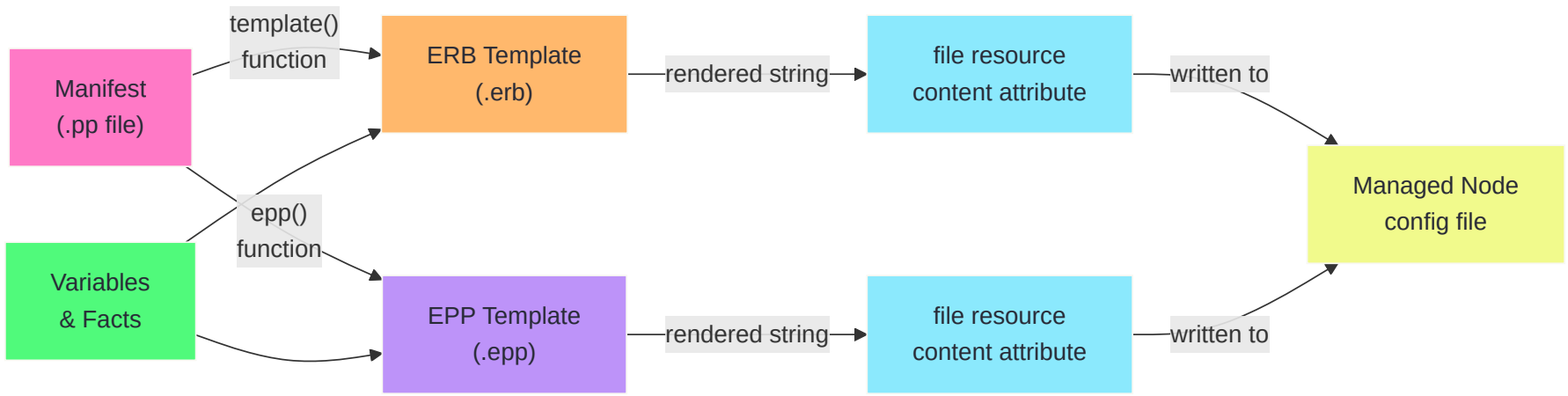
Bad approach: static files per environment

```
files/nginx.conf.production
files/nginx.conf.staging
files/nginx.conf.dev
# ...maintain three files whenever structure changes
```

Good approach: one template + Hiera data

```
templates/nginx.conf.epp
data/production.yaml  # worker_processes: 8
data/staging.yaml     # worker_processes: 2
```

Template Flow



ERB Templates — Syntax Overview

ERB (Embedded Ruby) is the older template format:

```
# templates/nginx.conf.erb

worker_processes <%= @worker_processes %>;
user             <%= @nginx_user %>;

events {
    worker_connections 1024;
}

http {
    <%- @listen_ports.each do |port| -%>
    listen <%= port %>;
    <%- end -%>

    server_name <%= @server_name %>;

    <% if @enable_gzip -%>
    gzip on;
    gzip_types text/plain text/css application/json;
    <% end -%>
```

ERB Tag Reference

Tag	Purpose
`<%= expr %>`	Output the value of an expression
`<% code %>`	Execute Ruby code (no output)
`<%- code -%>`	Execute Ruby code, strip surrounding whitespace
`<%# comment %>`	Template comment — not rendered

> In ERB templates, Puppet variables are accessed as Ruby instance variables with `@` prefix: `$config_dir` → `@config_dir`.

Using an ERB Template

```
class nginx (
  Integer $worker_processes = 2,
  String  $nginx_user       = 'www-data',
  Boolean $enable_gzip      = true,
  Array   $listen_ports     = [80],
  String  $server_name      = $facts['networking']['fqdn'],
) {
  file { '/etc/nginx/nginx.conf':
    ensure  => file,
    content => template('nginx/nginx.conf.erb'),  # <-- ERB
    require => Package['nginx'],
    notify  => Service['nginx'],
  }
}
```

> `template()` — takes the path `module_name/template_name.erb`.
> Variables available in the template = all variables in the current scope.

EPP Templates — Puppet-Native

EPP (Embedded Puppet) is the **modern, recommended** template format:

```
<%- | Integer $worker_processes,
      String  $nginx_user,
      Boolean $enable_gzip,
      Array   $listen_ports,
      String  $server_name
| -%>
# /etc/nginx/nginx.conf — managed by Puppet, do not edit manually

worker_processes <%= $worker_processes %>;
user             <%= $nginx_user %>;

http {
    <% $listen_ports.each |$port| { -%>
    listen <%= $port %>;
    <% } -%>

    server_name  <%= $server_name %>;

    <% if $enable_gzip { -%>
    gzip on;
    <% } -%>
```

EPP Tag Reference

Tag	Purpose
`<%= $expr %>`	Output value
`<% code %>`	Execute Puppet code
`<%- code -%>`	Execute, strip whitespace
`<%# comment %>`	Comment
`<%- | params | -%>`	Parameter list at template top

> EPP uses **Puppet syntax** — `$variables`, `if`, `each`, etc. No Ruby knowledge required.

Using an EPP Template

```
file { '/etc/nginx/nginx.conf':
  ensure  => file,
  content => epp('nginx/nginx.conf.epp', {
    worker_processes => $worker_processes,
    nginx_user       => $nginx_user,
    enable_gzip      => $enable_gzip,
    listen_ports     => $listen_ports,
    server_name      => $server_name,
  }),
  require => Package['nginx'],
  notify  => Service['nginx'],
}
```

> `epp()` — takes a hash of parameters. These become `$variables` inside the template. Only the declared parameters in the `| ... |` block are accessible.

Inline Templates

For short dynamic strings you can embed templates directly in manifests:

```
# Inline ERB
$motd_content = inline_template("Managed by Puppet\nHostname: <%= @hostname %>\nOS: <%= @osfamily %>\n")

# Inline EPP
$motd_content = inline_epp("Hostname: <%= \$facts['networking']['hostname'] %>\n")

file { '/etc/motd':
  ensure  => file,
  content => $motd_content,
}
```

> For anything beyond a line or two, use a proper template file — it's easier to read, test, and maintain.

ERB vs. EPP — Choosing the Right Format

	ERB	EPP
Variables	`@variable` (Ruby instance var)	`$variable` (Puppet)
Conditionals	Ruby `if/else/end`	Puppet `if {} else {}`
Loops	Ruby `.each do	x
Parameter declaration	None — all scope vars available	Explicit `
Requires	Ruby knowledge	Puppet knowledge only
Recommended	Legacy code	New development

Part 4 — Separation of Code and Data with Hiera

The Problem: Data Embedded in Code

```
# Bad: data hardcoded in a class
class ntp {
  $ntp_servers = ['ntp1.example.com', 'ntp2.example.com']
  $ntp_restrict = ['127.0.0.1', '::1']
  $drift_file = '/var/lib/ntp/drift'
  # ...
}
```

Problems:

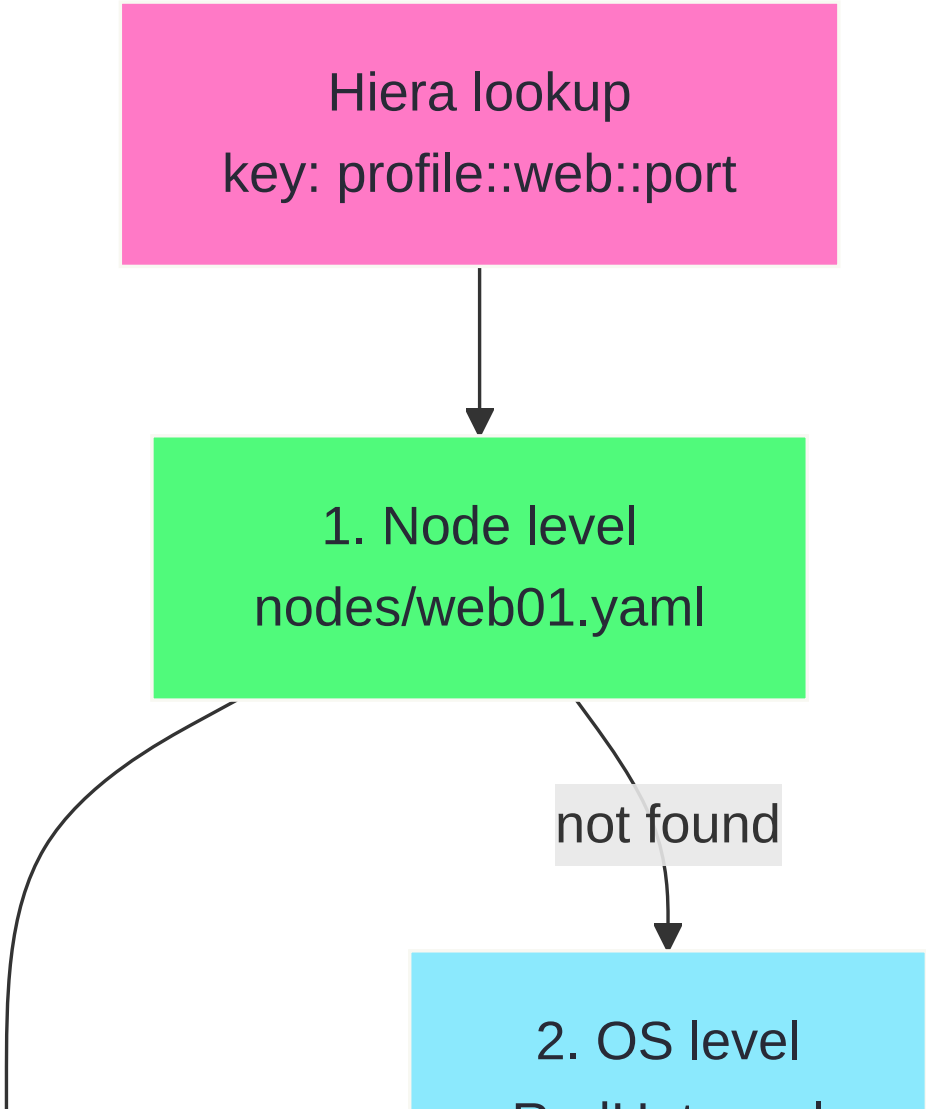
- Different NTP servers per environment require multiple classes or conditionals
- Code changes needed for every data variation
- Class cannot be reused across organizations without modification

The Solution — Hiera

Hiera = "hierarchy" — a key/value lookup system where the answer depends on **where you are in the hierarchy**.

```yaml
# /etc/puppetlabs/puppet/hiera.yaml  (global)
---
version: 5

hierarchy:
  - name: "Node-specific data"
    path: "nodes/%{trusted.certname}.yaml"

  - name: "OS-specific data"
    path: "os/%{facts.os.family}.yaml"

  - name: "Common data"
    path: "common.yaml"

defaults:
  datadir: "/etc/puppetlabs/code/environments/%{environment}/data"
```

Hiera Lookup Chain



Writing Hiera Data

```yaml
# data/common.yaml
---
ntp::servers:
  - 'pool.ntp.org'
  - '0.pool.ntp.org'

ntp::drift_file: '/var/lib/ntp/drift'

profile::base::packages:
  - vim
  - curl
  - htop
```

```yaml
# data/os/RedHat.yaml
---
ntp::servers:
  - 'rhel-ntp.example.com'

profile::base::packages:
```

Automatic Parameter Lookup (APL)

The most powerful Hiera feature: Puppet **automatically** looks up class parameters in Hiera using the key `classname::parameter`:

```
# class definition
class ntp (
  Array[String] $servers    = ['pool.ntp.org'],
  String        $drift_file = '/var/lib/ntp/drift',
) {
  # use $servers and $drift_file
}
```

```
# data/common.yaml
ntp::servers:
  - '0.de.pool.ntp.org'
  - '1.de.pool.ntp.org'

ntp::drift_file: '/var/lib/ntp/drift'
```

Explicit Lookup

When you need to look up data outside of a class parameter:

```
# Look up a single value
$admin_email = lookup('monitoring::admin_email')

# Look up with a default fallback
$log_level = lookup('app::log_level', { 'default_value' => 'info' })

# Look up with type validation
$max_threads = lookup('app::max_threads', Integer, 'first', 100)

# Merge arrays from all levels in the hierarchy
$base_packages = lookup('profile::base::packages', Array, 'unique')
```

Merge Strategies

Hiera supports different merge strategies for lookups that span multiple hierarchy levels:

Strategy	Behaviour	Use case
`first` (default)	Return value from highest priority level	Scalars — one answer wins
`unique`	Merge arrays, deduplicate	Package lists, port arrays
`hash`	Shallow merge of hashes	Config hashes, one level deep
`deep`	Deep recursive merge of hashes	Nested config hashes

```
# hiera.yaml — configure merge strategy per key
lookup_options:
```

Hiera in Practice — Environment Data

```
environments/production/
└── data/
    ├── common.yaml              # global defaults
    ├── os/
    │   ├── Debian.yaml          # Debian/Ubuntu overrides
    │   └── RedHat.yaml          # RHEL/Rocky overrides
    ├── nodes/
    │   ├── web01.example.com.yaml
    │   └── db01.example.yaml
    └── roles/
        ├── webserver.yaml
        └── database.yaml
```

> Keep your Hiera data files alongside your Puppet code in the control repository — version controlled, reviewed, and deployed together.

Inheritance via Hiera — Exceptions Made Elegant

The hierarchy naturally expresses **rules and exceptions** without code changes:

```yaml
# common.yaml — the rule
profile::ntp::servers:
  - 'pool.ntp.org'

# nodes/legacy-server.example.com.yaml — the exception
profile::ntp::servers:
  - 'old-ntp.legacy.example.com'
```

```yaml
# common.yaml
profile::firewall::allow_ssh_from: ['10.0.0.0/8']

# nodes/bastion.example.com.yaml
profile::firewall::allow_ssh_from: ['0.0.0.0/0']
```

> **Hiera replaces inheritance.** Managing exceptions in data is cleaner, safer, and more auditable than overriding in code.

`puppet lookup` — Debug Your Hierarchy

```
# Look up a key for a specific node
puppet lookup ntp::servers \
  --node web01.example.com \
  --explain

# Output:
# Searching for "ntp::servers"
#   Merge strategy: first
#   data/nodes/web01.example.com.yaml: key not found
#   data/os/Debian.yaml: key not found
#   data/common.yaml: Found value
```

> Always use `--explain` when debugging unexpected values — it shows exactly where in the hierarchy the value was found or not found.

Hiera Secret Management — eyaml

For secrets (passwords, API keys), use the **hiera-eyaml** backend:

```
# Install the gem
gem install hiera-eyaml
eyaml createkeys      # generates keys/

# Encrypt a value
eyaml encrypt -s 'my-secret-password'
# => ENC[PKCS7,MIIBeQYJKoZIhvcNA...]
```

```
# data/common.yaml
app::db_password: >
  ENC[PKCS7,MIIBeQYJKoZIhvcNA...]
```

> Encrypted values are decrypted transparently by the Puppet Server. Commit encrypted Hiera data safely to version control — the key files never leave the server.

Part 5 — Data Types, Custom Functions & Advanced Hiera

The Puppet Type System

Puppet has a rich, composable type system used to validate parameters at catalog compile time:

```
# Scalar types
String              # any string
Integer             # whole number
Float               # decimal number
Boolean             # true or false
Undef               # the null value (undef)

# Parameterised types
String[1]                   # non-empty string (minimum length 1)
Integer[1, 65535]           # valid port number
Array[String]               # array of strings only
Hash[String, Integer]       # hash with string keys, integer values

# Composable types
Optional[String]            # String or Undef
Variant[String, Integer]    # String or Integer
```

`Optional` — Nullable Parameters

```
# Without Optional — caller cannot pass undef
class myapp (
  String $owner,   # required; undef raises a type error
) { }

# With Optional — parameter truly optional
class myapp (
  Optional[String] $owner = undef,
) {
  if $owner {
    file { '/opt/myapp':
      ensure => directory,
      owner  => $owner,
    }
  }
}
```

> Use `Optional[Type]` whenever a parameter may legitimately be absent. Without it, Puppet rejects `undef` with a type mismatch error.

`Enum` — Restrict to an Allowed Set of Values

```
class myapp (
  Enum['debug', 'info', 'warn', 'error'] $log_level = 'info',
  Enum['running', 'stopped']             $ensure     = 'running',
) {
  file { '/etc/myapp/config.yaml':
    content => "log_level: ${log_level}\n",
  }
}
```

```
# Puppet rejects any value not in the list immediately:
# Expected Enum['debug', 'info', 'warn', 'error'], got String 'verbose'
class { 'myapp': log_level => 'verbose' }  # ← compile error
```

> The error surfaces at catalog compilation — not at runtime or in a log file three hours later.

`Variant` and `Pattern`

```
# Variant — accept multiple concrete types
class myapp (
  Variant[String, Array[String]] $admin_emails = 'admin@example.com',
) {
  # Normalise to array regardless of what was passed
  $email_list = $admin_emails ? {
    String => [$admin_emails],
    Array  => $admin_emails,
  }
}

# Pattern — validate string format against a regex
class firewall_rule (
  Pattern[/^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}(\/\d{1,2})?$/] $source_cidr,
) { }
```

`Struct` — Typed Hash Schema

Enforce the exact structure of a hash parameter:

```
class myapp (
  Struct[{
    host               => String[1],
    port               => Integer[1, 65535],
    name               => String[1],
    Optional[user]     => String,
    Optional[password] => Sensitive[String],
  }] $database,
) {
  file { '/etc/myapp/db.conf':
    content => epp('myapp/db.conf.epp', { db => $database }),
    mode    => '0600',
  }
}
```

```
# data/common.yaml
myapp::database:
```

Type Aliases

Define named, reusable types to DRY up parameter declarations:

```
# modules/mymodule/types/port.pp
type Mymodule::Port = Integer[1, 65535]

# modules/mymodule/types/loglevel.pp
type Mymodule::LogLevel = Enum['debug', 'info', 'warn', 'error', 'fatal']

# modules/mymodule/types/vhostconfig.pp
type Mymodule::VhostConfig = Struct[{
  servername        => String[1],
  port              => Mymodule::Port,
  document_root     => String[1],
  Optional[ssl]     => Boolean,
}]

# Use the aliases throughout the module
class mymodule (
  Mymodule::Port     $listen_port = 8080,
  Mymodule::LogLevel $log_level   = 'info',
```

Custom Puppet Functions — The 4.x Ruby API

Place functions in `lib/puppet/functions/<module>/<name>.rb`:

```ruby
# modules/mymodule/lib/puppet/functions/mymodule/validate_cidr.rb
Puppet::Functions.create_function(:'mymodule::validate_cidr') do
  dispatch :validate_cidr do
    param       'String',  :cidr
    return_type 'Boolean'
  end

  def validate_cidr(cidr)
    require 'ipaddr'
    IPAddr.new(cidr)
    true
  rescue ArgumentError
    false
  end
end
```

```ruby
# Use in a manifest
```

Custom Puppet Functions — Pure Puppet

Simple logic can be written as pure-Puppet functions — no Ruby needed:

```
# modules/mymodule/functions/ensure_array.pp
function mymodule::ensure_array(
  Variant[String, Array[String]] $value
) >> Array[String] {
  $value ? {
    String => [$value],
    Array  => $value,
  }
}
```

```
# modules/mymodule/functions/prefix_keys.pp
function mymodule::prefix_keys(
  Hash   $input,
  String $prefix,
) >> Hash {
  $input.reduce({}) |$memo, $pair| {
    $memo + { "${prefix}${pair[0]}" => $pair[1] }
```

stdlib — Essential String and Array Functions

`puppetlabs-stdlib` is the de-facto standard library for Puppet:

```
# Strings
stdlib::basename('/etc/nginx/nginx.conf')    # => 'nginx.conf'
stdlib::dirname('/etc/nginx/nginx.conf')     # => '/etc/nginx'
stdlib::strip('  hello  ')                   # => 'hello'
upcase('hello')                              # => 'HELLO'
downcase('WORLD')                            # => 'world'

# Arrays and hashes
stdlib::flatten([[1, 2], [3, 4]])            # => [1, 2, 3, 4]
stdlib::unique([1, 2, 2, 3])                # => [1, 2, 3]
stdlib::sort([3, 1, 2])                      # => [1, 2, 3]
stdlib::keys({'a' => 1, 'b' => 2})           # => ['a', 'b']
stdlib::values({'a' => 1, 'b' => 2})         # => [1, 2]
stdlib::merge({'a' => 1}, {'b' => 2})        # => {'a' => 1, 'b' => 2}
```

stdlib — Resource Helper Functions

```
# Declare packages without risk of duplicate resource errors
# (safe to call from multiple profiles)
ensure_packages(['vim', 'curl', 'git'], { 'ensure' => 'installed' })

# Declare a resource only if it hasn't already been declared
ensure_resource('file', '/opt/myapp', {
  'ensure' => 'directory',
  'owner'  => 'deploy',
  'mode'   => '0755',
})

# Type-safe assertion with a helpful error message
assert_type(Integer[1, 65535], $port) |$expected, $actual| {
  fail("${port} is not a valid port (expected ${expected}, got ${actual})")
}

# Validate strings match a pattern (older API, still widely used)
validate_re($env, '^(production|staging|development)$',
  "Environment must be production, staging, or development")
```

The Three Hiera Data Layers

Hiera evaluates three distinct layers, from highest to lowest priority:

```
Priority: high ──────────────────────────────────────────────► low

┌──────────────────────────────────────────────────────────────────┐
│ Environment layer  (environments/production/                     │
│                     hiera.yaml + data/)                          │
│ ← Most active. Your node/OS/common data lives here.              │
├──────────────────────────────────────────────────────────────────┤
│ Module layer       (modules/nginx/hiera.yaml +                   │
│                     modules/nginx/data/)                         │
│ ← Module author defaults. User data overrides these.             │
├──────────────────────────────────────────────────────────────────┤
│ Global layer       (/etc/puppetlabs/puppet/                      │
│                     hiera.yaml)                                  │
│ ← Site-wide or legacy. Migrated to env layer first.              │
└──────────────────────────────────────────────────────────────────┘
```

> For new deployments: keep all data in the **environment layer**. Reserve the global

Module-Level Hiera in Depth

Modules can ship their own hierarchy for defaults:

```yaml
# modules/nginx/hiera.yaml
---
version: 5
defaults:
  datadir: data
  data_hash: yaml_data
hierarchy:
  - name: "OS family"
    path: "os/%{facts.os.family}.yaml"
  - name: "Module defaults"
    path: "common.yaml"
```

```yaml
# modules/nginx/data/common.yaml
nginx::worker_processes: 2
nginx::enable_gzip: true
nginx::listen_ports:
  - 80
```

`lookup_options` — Per-Key Merge Configuration

Control how each key is merged across hierarchy levels:

```yaml
# data/common.yaml
---
lookup_options:
  # Always merge arrays — never first-wins
  profile::base::packages:
    merge: unique

  # Recursive deep merge for nested config hashes
  profile::app::config:
    merge: deep

  # Shallow hash merge — one level only
  profile::nginx::vhosts:
    merge: hash

# The actual data values follow normally
profile::base::packages:
  - curl
  - vim
```

Hiera Merge in Practice — Layered Package Lists

```yaml
# data/common.yaml
profile::base::packages:
  - curl
  - vim
  - git

# data/os/RedHat.yaml
profile::base::packages:
  - bash-completion
  - dnf-utils

# data/nodes/web01.example.com.yaml
profile::base::packages:
  - apache2-utils
```

```
# With merge: unique — all three lists are combined and deduplicated
$packages = lookup('profile::base::packages', Array, 'unique')
# => ['curl', 'vim', 'git', 'bash-completion', 'dnf-utils', 'apache2-utils']
```

eyaml — Key Management and Rotation

```
# Install and generate a keypair
gem install hiera-eyaml
eyaml createkeys
# → keys/private_key.pkcs7.pem
# → keys/public_key.pkcs7.pem

# Store private key on the Puppet Server ONLY
sudo mkdir -p /etc/puppetlabs/puppet/keys
sudo install -m 400 keys/private_key.pkcs7.pem \
  /etc/puppetlabs/puppet/keys/
sudo install -m 444 keys/public_key.pkcs7.pem \
  /etc/puppetlabs/puppet/keys/

# Encrypt a new secret
eyaml encrypt -s 'my-db-password' \
  --pkcs7-public-key=keys/public_key.pkcs7.pem

# Rotate keys: re-encrypt all secrets with a new keypair
eyaml recrypt --old-pkcs7-public-key=old.pem \
              --new-pkcs7-public-key=new.pem \
              data/common.eyaml
```

eyaml — hiera.yaml Configuration

```yaml
# hiera.yaml — add the eyaml backend
---
version: 5
defaults:
  datadir: "data"
hierarchy:
  - name: "Node secrets"
    lookup_key: eyaml_lookup_key
    path: "nodes/%{trusted.certname}.eyaml"
    options:
      pkcs7_private_key: /etc/puppetlabs/puppet/keys/private_key.pkcs7.pem
      pkcs7_public_key:  /etc/puppetlabs/puppet/keys/public_key.pkcs7.pem

  - name: "Common data"
    data_hash: yaml_data
    path: "common.yaml"
```

> The **public key** can be committed to version control — anyone can encrypt new secrets. The **private key** lives only on the Puppet Server and never leaves it.

Deferred Functions — Secrets at Apply Time

Deferred functions run **on the agent** at apply time, not on the server at compile time:

```
# The secret never touches the Puppet Server catalog
class myapp (
  Sensitive[String] $db_password = Deferred(
    'vault::secret',
    ['secret/myapp/database', 'password']
  ),
) {
  file { '/etc/myapp/db.conf':
    # Sensitive() prevents the value from appearing in logs
    content => Sensitive(epp('myapp/db.conf.epp', {
      password => $db_password,
    })),
    mode    => '0600',
  }
}
```


> Deferred functions integrate Puppet with **HashiCorp Vault, AWS Secrets**

`puppet lookup` — Advanced Debugging

```
# Find where a key resolves, with full trace
puppet lookup profile::nginx::worker_processes \
  --node web01.example.com \
  --explain \
  --environment production

# Force a specific merge strategy for ad-hoc testing
puppet lookup profile::base::packages \
  --merge unique \
  --node web01.example.com

# Compare values across environments
for env in production staging development; do
  echo "=== $env ===";
  puppet lookup ntp::servers --environment $env;
done

# Verify eyaml decryption works (runs on the server)
puppet lookup myapp::db_password \
  --node web01.example.com \
  --explain
```

Day 2 — Summary

Concept	Key takeaway
Iterators	`each` creates resources; `map` / `filter` / `reduce` transform data
Facts	System info collected by Facter; available as `$facts` hash
Custom facts	Ruby (`lib/facter/`) or external scripts/YAML (`facts.d/`)
Templates	ERB (legacy) and EPP (modern) — separate structure from values
Template functions	`template()` for ERB, `epp()` for EPP
Hiera	Hierarchical key/value store for class parameters and data
APL	Puppet auto-wires `classname::parameter` keys from Hiera

Day 2 — What's Next

Tomorrow we focus on the **professional craft** of Puppet:

- Module structure and the **Roles and Profiles** pattern
- Version control with Git and the **control repository** concept
- Managing modules and environments with **r10k**
- Quality assurance: **rspec-puppet**, code review, and the PDK

> Today's Hiera and template exercises are the foundation for tomorrow's module work.